

BACHELIER EN INFORMATIQUE
Stages d'intégration et d'activités professionnelles

CHIMPS LAB

Maitre de stage : Nicolas Santamaria

Gaucher Adrien

Année académique 2025-26

Remerciements

Je tiens à remercier l'ensemble des personnes qui m'ont accompagné durant mon stage au sein du Chimps Lab.

Je remercie tout particulièrement mon maître de stage pour son encadrement, sa disponibilité et ses conseils tout au long de ces trois mois. Son suivi régulier m'a permis de progresser aussi bien sur le plan technique que sur le plan méthodologique, et de mieux comprendre les exigences liées à un projet professionnel réel.

Enfin, je remercie mon établissement et l'ensemble des enseignants de ma formation, qui m'ont fourni les bases théoriques et pratiques nécessaires pour aborder ce stage dans de bonnes conditions. Ce stage représente une étape importante dans mon parcours, en me permettant de mettre en pratique les compétences acquises durant mon bachelier en informatique.

Table des matières

1. Introduction.....	1
2. Présentation de l'entreprise.....	1
2.1 Présentation générale.....	1
2.2 Organisation interne.....	2
2.3 Infrastructure.....	2
3. Contexte et objectifs du projet.....	3
3.1 Le projet.....	3
3.2 Objectifs.....	3
4. Organisation du stage et méthodologie de travail.....	4
5. Analyse du projet.....	5
5.1 État de l'art.....	5
5.2 Analyse des besoins.....	5
5.3 Choix d'architecture.....	7
5.3.1 Architecture centralisée.....	7
5.3.2 Authentification des serrures.....	7
5.3.3 Utilisation des QR codes.....	7
5.3.4 Communication client-serveur.....	8
5.4 Cas d'utilisation.....	8
6. Développement.....	9
6.1 Développement hardware.....	9
6.1.1 Développement matériel.....	9
6.1.2 Développement du firmware.....	10
6.1.3 Développement de l'interface graphique.....	11
6.2 Développement logiciel.....	12
6.2.1 Architecture générale.....	12
6.2.2 Base de données.....	12
6.2.3 Interface d'administration.....	13
6.2.4 Gestion des accès et communication avec les serrures.....	13
6.3 Tests et validation.....	14
6.3.1 Validation du firmware.....	14
6.3.2 Validation du backend.....	14
6.3.3 Validation du système complet.....	15
6.3.4 Gestion des cas d'erreur.....	15
7. Technologies et outils utilisés.....	16
8. Difficultés rencontrées et solutions apportées.....	17
8.1 Communication temps réel avec les serrures.....	17
8.2 Organisation des tâches FreeRTOS.....	17
8.3 Une configuration pensée pour l'utilisateur.....	18
8.4 Évolution du prototype matériel.....	18
8.5 Développement d'une couche graphique embarquée.....	19
9. Bilan et conclusion.....	20
9.1 Bilan du projet.....	20
9.2 Bilan personnel.....	21
10. Annexes.....	22
10.1 Diagramme de cas d'utilisations.....	22
10.2 Schéma électrique.....	23

10.3 Photo du prototype.....	23
10.4 Diagramme architecture firmware.....	24
10.5 Diagramme state machine firmware.....	25
10.6 Diagramme architecture générale.....	25
10.7 Diagramme ERD de la BDD.....	26
10.8 Visuel dashboard de la plateforme web.....	26
10.9 Diagramme de séquence 1.....	27
10.10 Diagramme de séquence 2.....	28
10.11 Diagramme de séquence 3.....	29

1.Introduction

Dans le cadre de mon stage, j'ai participé au développement d'un système de contrôle d'accès connecté destiné à la gestion d'espaces à accès temporaire. Ce projet m'a permis d'intervenir sur plusieurs aspects du développement, allant de la conception du prototype matériel au développement du logiciel embarqué et de la plateforme web.

L'objectif de ce rapport est de présenter le contexte dans lequel le projet a été réalisé, les besoins auxquels il devait répondre ainsi que les choix effectués au cours de sa conception et de son développement. Une attention particulière sera portée aux différentes étapes de réalisation du prototype et aux problématiques rencontrées au cours du projet.

Le rapport présente dans un premier temps l'entreprise et le contexte du projet, puis les besoins identifiés et les choix d'architecture retenus. Il décrit ensuite le développement des différents composants du système avant de revenir sur les résultats obtenus et les enseignements tirés de cette expérience.

2. Présentation de l'entreprise

2.1 Présentation générale

Chimps Lab est une structure technologique et créative qui combine développement logiciel, production multimédia et expérimentation technique. L'entreprise est centrée autour d'un environnement de travail polyvalent permettant de concevoir et tester des projets dans des domaines variés, allant du développement informatique à la création de contenus interactifs.

L'activité principale de Chimps Lab repose sur le développement de solutions logicielles sur mesure ainsi que sur des projets liés aux technologies immersives et interactives. L'entreprise intervient notamment dans des domaines tels que le développement de logiciels, les systèmes de gestion, ainsi que des projets liés aux moteurs de jeux vidéo / rendu 3D et aux environnements temps réel.

Le fondateur de la structure possède une expérience initiale en programmation graphique et en développement sur moteurs de jeu (notamment Unreal Engine). En parallèle, il développe des solutions logicielles orientées gestion, notamment pour des structures éducatives.

L'environnement de Chimps Lab est également particulier dans sa conception : il s'agit d'un espace hybride regroupant à la fois un espace de développement informatique, un studio équipé (fond vert, régie vidéo), ainsi que des espaces de travail collaboratif et des zones de réunion. Cette configuration permet de soutenir des projets techniques et créatifs dans un même lieu.

2.2 Organisation interne

Chimps Lab est une structure de petite taille, centrée principalement autour de son fondateur, qui assure à la fois les rôles de développeur, chef de projet et responsable technique. Selon les projets, des collaborateurs externes ou stagiaires peuvent être intégrés afin de renforcer l'équipe sur des tâches spécifiques.

Dans le cadre de mon stage, j'ai été encadré directement par le fondateur de l'entreprise, qui a assuré le rôle de maître de stage. Son suivi s'est fait de manière continue, avec des échanges réguliers permettant de valider l'avancement des tâches, d'orienter les choix techniques et de s'assurer de la cohérence globale du projet.

Le suivi des tâches se faisait de manière itérative, avec des points réguliers permettant d'ajuster les priorités en fonction de l'évolution du développement. La communication était directe et informelle, favorisant la réactivité et l'adaptation rapide aux besoins du projet.

2.3 Infrastructure

L'infrastructure du Chimps Lab repose sur un ensemble de technologies permettant le développement, l'hébergement et l'exploitation de différents services internes et applicatifs.

L'entreprise s'appuie sur une architecture distribuée composée de plusieurs services distincts. Cette séparation permet d'isoler les différents usages tels que le développement, la gestion de projet, le déploiement applicatif ou encore les traitements plus spécifiques comme le rendu ou le streaming.

Plusieurs serveurs sont utilisés au sein de l'infrastructure interne, chacun ayant un rôle dédié. Certains sont utilisés pour l'hébergement des outils de développement et de collaboration, comme Git et Planka, tandis que d'autres sont dédiés au développement ou aux traitements plus lourds, notamment liés à la vidéo ou à l'image.

Cette organisation permet une meilleure répartition des charges et une plus grande flexibilité dans l'utilisation des ressources.

Sur le plan du développement, le code source est géré via Git, tandis que la gestion de projet est assurée à l'aide de Planka. L'ensemble contribue à un environnement de travail structuré et cohérent avec les besoins des projets techniques de l'entreprise.

3. Contexte et objectifs du projet

3.1 Le projet

Le projet auquel j'ai participé consiste en la conception d'un système de contrôle d'accès connecté destiné à être intégré à une plateforme de gestion et de réservation d'espaces.

Le périmètre du projet porte principalement sur la gestion des accès physiques et sur les serrures connectées. L'objectif est de permettre à un utilisateur disposant d'un accès valide d'ouvrir automatiquement une porte pendant la période autorisée, sans intervention humaine.

Le système comprend une partie logicielle permettant de gérer les accès et les serrures, ainsi qu'une partie embarquée reposant sur un dispositif connecté capables de contrôler physiquement le verrouillage d'une porte.

Le projet vise ainsi à fournir une solution de contrôle d'accès autonome, administrable à distance et adaptable à différents types d'espaces professionnels.

3.2 Objectifs

Au-delà de la réalisation d'un prototype fonctionnel, le projet avait pour objectif de valider la faisabilité technique d'une solution complète de contrôle d'accès connecté.

Le premier objectif était de concevoir un système permettant de remplacer les badges physiques par des autorisations numériques et de contrôler automatiquement l'accès à différents espaces selon les droits accordés.

Le prototype devait également proposer une architecture suffisamment évolutive pour permettre, à terme, l'ajout de nouvelles fonctionnalités, de nouvelles serrures ou l'intégration avec différents systèmes de gestion.

Un autre objectif était d'expérimenter différentes approches techniques afin d'identifier les solutions les plus adaptées aux contraintes du projet, aussi bien pour la partie logicielle que pour le système embarqué.

4. Organisation du stage et méthodologie de travail

Tout au long du stage, l'organisation du travail s'est inspirée des méthodes agiles, tout en restant adaptée à la taille de l'entreprise et à l'équipe de développement. L'objectif était de favoriser une communication régulière, un suivi continu de l'avancement et une adaptation rapide aux besoins du projet.

chaque journée débutait par un court échange avec mon maître de stage. Ce point quotidien permettait de faire le bilan du travail réalisé la veille, de définir les objectifs de la journée et de discuter des éventuelles difficultés rencontrées.

En complément de ces points quotidiens, une réunion hebdomadaire était organisée avec le maître de stage afin de faire le bilan des travaux réalisés, d'évaluer l'avancement du projet et de définir les priorités pour les jours suivants. Ces réunions constituaient également un moment privilégié pour discuter des choix d'architecture, des solutions techniques envisagées et des améliorations possibles.

La gestion du code source était assurée à l'aide de Git, hébergé sur une instance Forgejo installée sur l'infrastructure interne de l'entreprise. Cette plateforme permettait de centraliser l'ensemble des dépôts du projet, de suivre l'historique des modifications et de faciliter la collaboration. Les nouvelles fonctionnalités étaient développées sur des branches dédiées avant d'être intégrées au projet principal après relecture.

Le suivi des tâches était réalisé à l'aide de Planka, un outil de gestion de projet inspiré de la méthode Kanban. Chaque fonctionnalité, correction de bug ou amélioration faisait l'objet d'une carte décrivant le travail à réaliser. Les tâches étaient organisées selon leur état d'avancement (*À faire, En cours, Terminé*), offrant une vision claire de la progression du projet.

Afin d'assurer une meilleure traçabilité, chaque tâche était également associée à une branche Git identifiée par un numéro de version (par exemple *V001*, *V002*, etc.). Les cartes étaient en outre catégorisées selon leur scope dans le projet (firmware, backend..). Cette organisation facilitait le suivi des développements, la gestion des priorités ainsi que le lien entre les évolutions du code et les fonctionnalités implémentées.

Le tableau était mis à jour aussi bien par mon maître de stage que par moi-même, ce qui permettait de suivre l'avancement du projet en temps réel et d'assurer une bonne coordination tout au long du développement.

Cette méthodologie de travail m'a permis d'évoluer dans un environnement proche des pratiques professionnelles actuelles. L'utilisation conjointe d'un système de gestion de versions, d'un outil de suivi des tâches et de réunions régulières a favorisé une organisation efficace, une meilleure communication et un développement progressif des différentes fonctionnalités du projet.

5. Analyse du projet

5.1 État de l'art

Le contrôle d'accès numérique repose aujourd'hui sur plusieurs approches courantes. Les badges RFID/NFC, très utilisés dans les environnements professionnels, nécessitent la distribution d'un support physique, ce qui est moins adapté à un accès ponctuel lié à une réservation.

Les serrures connectées grand public, comme Nuki ou August, proposent des fonctionnalités proches de celles recherchées dans ce projet. Elles sont cependant conçues comme des produits autonomes destinés à couvrir un ensemble de cas d'utilisation génériques. Notre approche vise au contraire à intégrer directement le contrôle d'accès dans une plateforme de réservation, avec un fonctionnement et un modèle de données définis spécifiquement pour ce contexte.

Notre objectif était de développer une solution directement adaptée à notre plateforme de réservation, tout en conservant la maîtrise de son fonctionnement. Le choix du QR code permet à l'utilisateur de présenter son pass depuis son smartphone, sans matériel supplémentaire.

Un autre objectif important était de ne pas dépendre d'un écosystème propriétaire pour le contrôle d'accès. Le développement de notre propre infrastructure permet de maîtriser le fonctionnement du système ainsi que le stockage des données liées aux utilisateurs, aux accès et aux événements.

Le prototype se positionne ainsi comme une solution de contrôle d'accès développée spécifiquement pour un système de réservation d'espaces professionnels, avec une maîtrise du matériel, du logiciel et des données.

5.2 Analyse des besoins

Acteurs du système

- **Administrateur** — gère les zones, les serrures et les pass depuis l'interface web ; consulte l'historique des événements et diagnostique les anomalies.
- **Utilisateur final** — dispose d'un pass d'accès (QR code) associé à une réservation ; n'interagit avec le système qu'au moment de présenter son QR code devant une serrure.
- **Serrure connectée** — dispositif embarqué autonome qui valide les demandes d'accès auprès du serveur central et exécute l'ouverture physique.

Besoin	Description
Gestion des zones	Créer, modifier et organiser des zones représentant des espaces physiques distincts, chacune regroupant une ou plusieurs serrures
Génération de pass	Générer un QR code unique (UUID) associé à une ou plusieurs zones, avec période de validité définie
Activation / désactivation d'un pass	Permettre la désactivation ou réactivation manuelle d'un pass avant l'expiration de sa période de validité
Validation d'accès	Vérifier, à chaque présentation d'un QR code, l'existence du pass, sa validité temporelle et son association avec la zone concernée
Initialisation d'une serrure	Associer un dispositif physique à une zone lors de sa mise en service, via un processus de configuration dédié
Journalisation	Enregistrer chaque tentative d'accès, ouverture réussie ou refus dans un journal d'événements consultable par l'administrateur
Suivi en temps réel	Refléter instantanément l'état du système et des événements sur le tableau de bord d'administration
Ouverture a distance	Permettre à un administrateur de déclencher l'ouverture d'une serrure à distance afin de conserver un contrôle étendu sur les accès.

Besoins non fonctionnels

Critère	Exigence
Sécurité	Authentifier les communications entre les serrures et le serveur à l'aide d'un token afin de limiter les accès non autorisés.
Performance	Garantir un temps de réponse suffisamment faible pour permettre une ouverture quasi immédiate après validation de l'accès.
Fiabilité	Assurer un fonctionnement stable de la serrure et une gestion correcte des erreurs, notamment en cas de perte ou d'instabilité du réseau.
Disponibilité	Permettre à la serrure de reprendre automatiquement son fonctionnement après une coupure réseau ou un redémarrage.
Maintenabilité	Concevoir le système de manière suffisamment modulaire pour faciliter son évolution et le remplacement de ses différents composants.
Utilisabilité	Permettre la configuration et l'utilisation de la serrure sans nécessiter de connaissances techniques particulières de la part de l'utilisateur.

5.3 Choix d'architecture

Après avoir identifié les besoins fonctionnels et non fonctionnels, plusieurs choix d'architecture ont été réalisés afin de concevoir un système répondant aux contraintes du projet. Ces choix concernent principalement la gestion des accès, les mécanismes d'authentification, le support d'identification des utilisateurs ainsi que le mode de communication entre les différents composants.

5.3.1 Architecture centralisée

Le prototype repose sur une architecture centralisée dans laquelle l'ensemble des droits d'accès est stocké sur un serveur unique. Les serrures ne conservent localement que leur configuration (ip du serveur, token d'authentification et paramètres réseau) et sollicitent le serveur pour chaque demande d'ouverture.

Cette approche présente plusieurs avantages. Elle permet tout d'abord la révocation immédiate d'un droit d'accès sans nécessiter de synchronisation avec les serrures déjà déployées. Elle simplifie également l'administration du système en regroupant l'ensemble des informations au sein d'une base de données unique. Enfin, elle facilite la journalisation des événements en conservant un historique complet de toutes les tentatives d'accès.

En contrepartie, cette architecture introduit une dépendance à la disponibilité du réseau et du serveur central. Une interruption de la communication empêche la validation des nouvelles demandes d'accès. Cette limitation a été jugée acceptable dans le cadre du prototype.

5.3.2 Authentification des serrures

Chaque serrure possède un token d'authentification généré lors de son enregistrement dans le système. Ce token est transmis à la serrure lors de sa configuration initiale puis utilisé pour toutes les communications avec le serveur.

L'utilisation d'un token permet au serveur de vérifier l'identité de la serrure à l'origine de chaque requête. La validation d'un accès ne repose donc pas uniquement sur le QR code présenté par l'utilisateur : le serveur vérifie également que la demande provient bien d'un dispositif enregistré et autorisé. Cette double vérification renforce la sécurité du système et limite les risques d'usurpation.

5.3.3 Utilisation des QR codes

Plusieurs technologies d'identification auraient pu être envisagées, notamment les badges RFID/NFC, le Bluetooth ou encore les claviers à code.

Le QR code a été retenu car il ne nécessite aucun matériel spécifique côté utilisateur. Un simple smartphone suffit pour présenter un pass d'accès. Cette solution simplifie également la distribution

des accès temporaires, puisqu'un QR code peut être généré et transmis numériquement sans intervention physique.

Le même principe a également été appliqué à la configuration des serrures. Plutôt que d'utiliser une interface de configuration complexe ou une connexion filaire, les paramètres réseau et les informations d'authentification sont transmis au moyen de QR codes affichés depuis l'interface d'administration. Le QR code devient ainsi une véritable interface homme-machine (IHM), simplifiant considérablement l'installation des dispositifs.

5.3.4 Communication client-serveur

Chaque demande d'accès suit un modèle de communication de type requête/réponse. Lorsqu'un utilisateur présente un QR code, la serrure transmet les informations nécessaires au serveur, qui réalise l'ensemble des vérifications avant de renvoyer sa décision.

Ce fonctionnement garantit que toutes les décisions d'autorisation sont prises à partir des données les plus récentes disponibles dans la base de données. Les modifications apportées aux utilisateurs, aux droits d'accès ou aux serrures sont ainsi prises en compte immédiatement, sans mécanisme de synchronisation supplémentaire entre les différents équipements.

5.4 Cas d'utilisation

Afin de formaliser les interactions entre les différents acteurs et le système, un diagramme de cas d'utilisation a été réalisé (voir en annexe). Celui-ci met en évidence les principales fonctionnalités offertes par la plateforme, aussi bien pour les administrateurs que pour les utilisateurs finaux et les serrures connectées.

L'administrateur assure la gestion des zones, des serrures et des passes d'accès. Il peut également consulter les événements enregistrés et déclencher l'ouverture d'une serrure à distance lorsque cela est nécessaire.

L'utilisateur final interagit uniquement avec la serrure en présentant son QR code afin de demander l'accès à un espace réservé.

Enfin, la serrure connectée constitue un acteur à part entière du système. Elle est responsable de son initialisation, de la transmission des demandes d'accès au serveur, de l'exécution des commandes d'ouverture ainsi que de la notification de sa réinitialisation.

L'analyse des besoins et les choix d'architecture présentés dans cette section ont permis de définir le périmètre fonctionnel et technique du prototype. La suite de ce rapport détaille sa réalisation concrète, en commençant par le développement de la partie matérielle et du firmware embarqué, avant d'aborder le développement logiciel du backend et de l'interface d'administration, puis les tests réalisés pour valider le fonctionnement de l'ensemble du système.

6. Développement

6.1 Développement hardware

6.1.1 Développement matériel

Le développement de la partie matérielle a consisté à concevoir un prototype de serrure connectée capable d'identifier un QR code, de communiquer avec le serveur central et de commander l'ouverture d'une serrure électromagnétique.

Dans un premier temps, l'ensemble du prototype a été réalisé sur une plaque d'essai (*breadboard*). Cette approche a permis de modifier facilement le câblage au fur et à mesure de l'avancement du développement, de remplacer rapidement certains composants en cas de besoin et de faciliter les phases de débogage.

Le cœur du système repose sur un microcontrôleur ESP32-CAM, retenu pour sa connectivité Wi-Fi intégrée ainsi que pour la présence d'une caméra permettant la lecture des QR codes sans nécessiter de matériel supplémentaire. Le choix de ce microcontrôleur s'appuie également sur ses caractéristiques matérielles, notamment sa capacité de calcul, son architecture multicœur et sa quantité de mémoire disponible. Ces ressources sont particulièrement adaptées au projet, qui doit gérer simultanément l'acquisition et le traitement des images, les communications réseau ainsi que les différentes tâches du système embarqué. Le décodage des QR codes est assuré à l'aide de la bibliothèque ESP32QRCodeReader, qui exploite directement les images acquises par la caméra afin d'extraire les informations d'identification.

La serrure est commandée par une alimentation de contrôle LIBO Access Control Power Supply K80 (12 V / 3 A), spécialement conçue pour les systèmes de contrôle d'accès. La serrure électromagnétique est maintenue sous tension en fonctionnement normal afin de rester verrouillée. Lorsqu'une demande d'ouverture est validée par le serveur, l'alimentation de la serrure est momentanément interrompue, ce qui libère le mécanisme de verrouillage pendant une durée déterminée avant le retour à l'état sécurisé.

Afin de faciliter les interactions avec l'utilisateur, le prototype intègre un écran OLED connecté à l'ESP32. Celui-ci affiche les différents états du système, tels que le démarrage, la connexion au réseau Wi-Fi, l'attente d'un QR code ou encore le résultat d'une demande d'ouverture. L'interface matérielle est volontairement limitée afin de réduire la complexité du dispositif.

Une fois le fonctionnement du prototype validé, une seconde version a été réalisée sur un circuit imprimé soudé. Les différents composants ont ensuite été intégrés dans un boîtier fixé directement sur une porte afin de reproduire des conditions d'utilisation proches d'un environnement réel. Cette étape a permis de passer d'un prototype de développement facilement modifiable à une version plus robuste et représentative du dispositif final.

6.1.2 Développement du firmware

Le firmware embarqué constitue le cœur du système de contrôle d'accès. Il est responsable de l'ensemble des traitements réalisés par la serrure connectée, depuis la lecture d'un QR code jusqu'à la commande d'ouverture de la serrure électromagnétique, en passant par les communications avec le serveur central. Son développement a été réalisé en langage C en s'appuyant sur le système d'exploitation temps réel FreeRTOS.

Le choix de FreeRTOS répond à la nécessité de gérer plusieurs traitements indépendants de manière concurrente. Un système de contrôle d'accès doit être capable d'assurer simultanément l'acquisition des QR codes, l'affichage de l'état du dispositif, l'exécution de services annexes ainsi que la logique de contrôle du système, sans qu'une opération longue ou bloquante ne perturbe l'ensemble de l'application.

L'architecture logicielle repose sur quatre tâches principales exécutées en parallèle (*voir diagramme en annexe*). Chacune possède une responsabilité clairement définie et ne réalise qu'un ensemble limité d'opérations. Cette séparation des responsabilités permet de limiter le couplage entre les différents modules et d'obtenir une architecture plus modulaire.

La première tâche est dédiée à la gestion de la caméra. Elle pilote le capteur de l'ESP32-CAM, réalise l'acquisition des images et décode les QR codes. Lorsqu'un QR code est reconnu, la tâche ne prend aucune décision concernant sa validité. Elle génère simplement un événement `EVENT_QR_READ` contenant les informations extraites avant de le transmettre à la file de messages.

Une seconde tâche est consacrée à la gestion de l'interface utilisateur. Elle pilote l'écran OLED et affiche en permanence l'état courant du dispositif. Les différentes phases du fonctionnement, telles que le démarrage, la configuration, la connexion au serveur, l'attente d'un QR code ou encore le résultat d'une demande d'accès, sont ainsi présentées à l'utilisateur sans impacter les autres traitements réalisés par le firmware.

Une troisième tâche exécute un serveur web embarqué. Celui-ci est utilisé pour certaines opérations spécifiques de maintenance ou d'administration du dispositif (comme l'ouverture à distance ou le reset de la serrure).

La dernière tâche correspond à la boucle principale (*Main Loop*), qui constitue le véritable orchestrateur du système. Contrairement aux autres tâches, elle n'est pas associée à un périphérique matériel particulier. Son rôle consiste à centraliser l'ensemble des traitements décisionnels du firmware et à coordonner les différents modules.

Afin de limiter les dépendances entre les tâches, celles-ci ne communiquent jamais directement entre elles. Chaque module produit uniquement des événements représentant les actions qu'il détecte, comme la lecture d'un QR code, la réception d'une commande ou la fin d'une opération. Ces événements sont insérés dans une file de messages (*Event Queue*) fournie par FreeRTOS. Fonctionnant selon le principe FIFO (*First In, First Out*), cette file constitue l'unique point de communication entre les différents composants du firmware.

La Main Loop consomme en permanence les événements présents dans cette file. Chaque événement est interprété puis transmis à une machine à états finis (*Finite State Machine*, voir *diagramme en annexe*), responsable de déterminer le comportement du système. Cette organisation permet de centraliser l'ensemble de la logique métier dans un seul composant, tandis que les autres tâches restent entièrement dédiées à leur domaine fonctionnel.

La machine à états finis constitue le composant central de décision du firmware. Elle représente les différents états dans lesquels peut se trouver la serrure et détermine les transitions à effectuer en fonction des événements reçus depuis l'Event Queue.

Lors du démarrage, elle permet notamment d'orienter le dispositif vers les différents états nécessaires à son initialisation, puis vers son état de fonctionnement normal lorsque les conditions requises sont réunies. En fonctionnement normal, les événements produits par les différentes tâches sont interprétés par la machine à états, qui détermine le traitement à effectuer.

6.1.3 Développement de l'interface graphique

Pour rappel, l'interface utilisateur de la serrure est assurée par un écran OLED permettant d'afficher les différentes étapes du fonctionnement du prototype, telles que la configuration initiale, les messages d'état ou le résultat d'une demande d'accès.

Lors des premières phases de développement, l'affichage reposait directement sur la bibliothèque U8g2, utilisée comme pilote de l'écran OLED. Si cette bibliothèque permet de piloter efficacement l'affichage, elle se limite essentiellement à des primitives de dessin de bas niveau et ne fournit pas d'outils facilitant le développement d'interfaces graphiques plus élaborées.

Afin de disposer d'un environnement de développement graphique plus riche, un portage partiel de la bibliothèque raylib a été réalisé pour l'ESP32. L'objectif n'était pas uniquement de remplacer les fonctions de dessin de U8g2, mais surtout de bénéficier de l'écosystème proposé par raylib, en particulier de la bibliothèque raymath.

L'utilisation de raymath a permis d'employer des structures et des opérations mathématiques de plus haut niveau, telles que les vecteurs, les matrices ou les quaternions, simplifiant ainsi les calculs nécessaires au développement de l'interface graphique. Cette approche offre une abstraction plus moderne et facilite l'implémentation d'animations, de transformations géométriques ou de futurs éléments graphiques plus évolués.

6.2 Développement logiciel

6.2.1 Architecture générale

Le backend constitue le point central du système de contrôle d'accès. Il assure l'interface entre les différents composants de la plateforme : le tableau de bord d'administration, les serrures connectées ainsi que la base de données. Il centralise notamment la gestion des droits d'accès et la validation des demandes d'ouverture, afin de garantir un comportement cohérent entre les différents dispositifs.

Le serveur a été développé avec Node.js en utilisant le framework Express. Ce choix fournit un environnement adapté au développement d'une API REST et à la gestion des communications avec les différents composants du système. Afin d'organiser le code de manière claire et évolutive, une architecture inspirée du modèle MVC (Model - View - Controller) a été adoptée. Les routes assurent le routage des requêtes, les contrôleurs orchestrent les traitements métier tandis que les modèles, implémentés avec Mongoose, assurent les interactions avec la base de données.

Les échanges entre les différents composants reposent sur une API REST utilisant le format JSON. Cette approche permet d'uniformiser les communications entre des clients de nature différente, qu'il s'agisse du tableau de bord web ou du firmware embarqué sur les ESP32. Le backend constitue ainsi une interface commune entre la partie logicielle d'administration et les dispositifs physiques déployés sur le terrain.

6.2.2 Base de données

La persistance des données est assurée par MongoDB, une base de données orientée documents utilisée au travers de Mongoose. Ce choix s'est révélé particulièrement adapté au projet en raison de la structure des données manipulées et de leur proximité avec le format JSON utilisé par l'API.

Chaque ressource principale de l'application est représentée par une collection dédiée.

L'utilisation de Mongoose permet de définir les modèles de données ainsi que leurs règles de validation, tout en simplifiant les opérations de lecture et d'écriture dans la base. Cette organisation facilite également les évolutions futures du projet, l'ajout de nouvelles propriétés ne nécessitant que peu de modifications de la structure existante.

Des scripts d'import et d'export ont également été mis en place afin de faciliter la sauvegarde, la restauration et le transfert des données. Des outils de migration comme mongo-migrate permettent quant à eux de faire évoluer la structure des données existantes lors des modifications du modèle.

6.2.3 Interface d'administration

Afin de faciliter l'administration du système, une interface web a été développée avec Vue.js. Celle-ci permet de centraliser l'ensemble des opérations de gestion sans nécessiter d'accès direct à la base de données.

Depuis cette interface, un administrateur peut créer et gérer les utilisateurs, enregistrer de nouvelles serrures, organiser celles-ci en zones, générer les QR codes de configuration destinés à leur installation ainsi que consulter l'historique des événements produits par le système.

L'ensemble des opérations réalisées depuis le tableau de bord transite exclusivement par l'API du backend. Cette organisation garantit que toutes les règles métier sont appliquées de manière uniforme.

6.2.4 Gestion des accès et communication avec les serrures

Le backend est responsable de toute la logique liée au contrôle d'accès. Lorsqu'une nouvelle serrure est créée depuis le tableau de bord, le serveur génère automatiquement son identifiant ainsi qu'un token d'authentification unique. Ces informations sont ensuite regroupées dans un QR code de configuration destiné à être scanné lors de l'installation de la serrure.

Au premier démarrage, le firmware détecte l'absence de configuration et passe automatiquement en mode initialisation. Après lecture du QR code, les paramètres de connexion ainsi que le token sont enregistrés dans la mémoire non volatile du microcontrôleur. Cette opération ne doit être réalisée qu'une seule fois, les informations étant ensuite conservées entre les redémarrages.

Les serrures utilisent un token propre à chaque dispositif plutôt qu'un mécanisme d'authentification basé sur des JWT. Une serrure possédant une identité permanente connue du backend, un simple token aléatoire est suffisant pour authentifier chacune de ses requêtes. Cette solution réduit la complexité du firmware tout en restant adaptée aux besoins du projet.

Lorsqu'un utilisateur présente un QR code devant la serrure, celui-ci est décodé puis transmis au backend via une requête HTTPS authentifiée. Le serveur vérifie alors la validité du QR code, sa période d'utilisation ainsi que les autorisations associées avant de renvoyer une réponse au firmware. Si l'accès est autorisé, la serrure est déverrouillée pendant la durée prévue ; dans le cas contraire, la demande est simplement refusée.

Chaque opération est enregistrée dans la collection Events, permettant de conserver un historique complet des accès, des refus et des différentes actions réalisées sur le système. Cette centralisation de la logique métier garantit que toutes les décisions sont prises par le backend, tandis que les serrures restent limitées à l'exécution des actions qui leur sont demandées.

6.3 Tests et validation

6.3.1 Validation du firmware

Les premiers tests ont porté sur le bon fonctionnement du firmware embarqué. Chaque fonctionnalité a été validée indépendamment avant d'être intégrée au reste du système.

Les essais réalisés ont notamment permis de vérifier :

- le démarrage du microcontrôleur ;
- le passage automatique en mode configuration lors de l'absence de paramètres ;
- la lecture des QR codes de configuration ;
- la sauvegarde des paramètres dans la mémoire NVS ;
- la reconnexion automatique après un redémarrage ;
- la lecture des QR codes utilisateurs ;
- le pilotage de la serrure électromagnétique.

Ces essais ont permis de valider le comportement de la machine à états ainsi que les interactions entre les différentes tâches FreeRTOS.

6.3.2 Validation du backend

Le backend a été testé au moyen de requêtes HTTP ainsi qu'à travers l'utilisation du tableau de bord d'administration.

Les principaux tests réalisés sont les suivants :

- Création d'une serrure
- Generation des QR codes de configurations
- Validation / refus des QR codes utilisateurs
- Enregistrement des événements
- Consultation des logs des événements

Ces tests ont permis de vérifier le bon fonctionnement de l'API ainsi que la cohérence des données enregistrées dans la base MongoDB.

6.3.3 Validation du système complet

Une fois les différents composants validés individuellement, plusieurs essais ont été réalisés afin de vérifier le fonctionnement global du prototype.

Le scénario de test principal est le suivant :

1. Création d'une nouvelle serrure depuis le tableau de bord.
2. Génération du QR code de configuration.
3. Installation de la serrure et lecture du QR.
4. Connexion au réseau Wi-Fi.
5. Connexion au backend.
6. Présentation d'un QR code utilisateur.
7. Validation des droits d'accès.
8. Ouverture de la serrure.
9. Enregistrement de l'événement.
10. Mise à jour du tableau de bord en temps réel.

Ce scénario a permis de valider l'ensemble de la chaîne de traitement, depuis la configuration initiale jusqu'à l'ouverture physique de la serrure.

6.3.4 Gestion des cas d'erreur

Des essais ont également été réalisés afin de vérifier le comportement du système en présence de situations anormales.

Les cas suivants sont gérés par le système :

- QR code utilisateur invalide
- Perte de connexion wifi
- Backend indisponible
- Redémarrage de l'ESP32
- Token de serrure invalide
- QR code de configuration invalide

Ces différents essais ont permis de vérifier que le système restait dans un état cohérent même en présence d'erreurs de communication ou de configuration.

7. Technologies et outils utilisés

Le développement du prototype a mobilisé différents outils logiciels et matériels couvrant le développement embarqué, le développement web ainsi que la gestion du projet. Le tableau ci-dessous présente les principales technologies utilisées au cours du stage.

Catégorie	Technologies / Outils
Langages	C, JavaScript, TypeScript
Développement embarqué	Arduino framework, FreeRTOS, PlatformIO
Backend	Node.js, Express
Frontend	Vue.js
Base de données	MongoDB
Communication	REST, HTTP, WebSocket
Gestion de versions	Git, Forgejo
Gestion de projet	Planka
Tests API	Insomnia
Environnement de développement	Visual Studio Code
Système d'exploitation	Linux
Matériel	ESP32-CAM, écran OLED, serrure électromagnétique, alimentation LIBO K80, breadboard, PCB à pastilles

Les principales bibliothèques ou utilisées au cours du développement sont présentées dans le tableau suivant.

Bibliothèque	Rôle
ESP32QRCodeReader	Acquisition et décodage des QR codes
ArduinoJson	Sérialisation et désérialisation des données JSON
raylib (portage)	Couche d'abstraction graphique développée pour le prototype
U8g2	Pilotage de l'écran OLED
Fetch	Requêtes HTTP du frontend vers le backend
Socket.IO	Communication temps réel entre le backend et le frontend
Mongoose	Modélisation des données MongoDB

L'utilisation de ces technologies a permis de développer une architecture complète couvrant l'ensemble des composants du système, depuis le firmware embarqué jusqu'à l'interface d'administration, tout en facilitant la maintenance et l'évolution du prototype.

8. Difficultés rencontrées et solutions apportées

Le développement du prototype a nécessité plusieurs ajustements techniques au cours du stage. Les difficultés rencontrées ont principalement concerné les contraintes matérielles de l'ESP32, l'organisation du firmware ainsi que l'amélioration progressive de l'expérience utilisateur. Chaque difficulté a conduit à une réflexion sur l'architecture du système et, dans plusieurs cas, à une évolution des choix de conception initiaux.

8.1 Communication temps réel avec les serrures

L'une des premières pistes explorées consistait à utiliser une communication WebSocket entre le backend et les serrures afin de maintenir une connexion persistante. Cette approche permettait notamment au backend d'envoyer directement des commandes aux dispositifs, sans avoir à établir une nouvelle connexion pour chaque échange.

Cependant, l'intégration d'une connexion WebSocket permanente sur l'ESP32 s'est révélée trop coûteuse en ressources dans l'architecture embarquée retenue. La coexistence de cette connexion avec les différentes tâches FreeRTOS augmentait notamment l'utilisation de la mémoire et la complexité de gestion du système, pouvant entraîner des instabilités.

Plusieurs implémentations ont été expérimentées, notamment Socket.IO puis la bibliothèque `esp_websocket_client` de Arduino. Bien que ces solutions fonctionnent sur des cas simples, leur utilisation conjointe avec les autres composants du firmware, en particulier la caméra, les différentes tâches FreeRTOS et la pile réseau, augmentait significativement la consommation des ressources de l'ESP32. La gestion d'une connexion WebSocket permanente introduisait également une complexité supplémentaire dans la gestion des états de connexion.

Après plusieurs essais, cette architecture a été abandonnée au profit d'un modèle plus simple dans lequel les serrures initient elles-mêmes les communications via des requêtes HTTP. Les WebSockets ont été conservés uniquement pour la communication entre le backend et l'interface d'administration, où ils permettent une mise à jour instantanée du tableau de bord sans impacter les contraintes matérielles des dispositifs embarqués.

8.2 Organisation des tâches FreeRTOS

Le firmware repose sur plusieurs tâches exécutées en parallèle, notamment pour la gestion de la caméra, de l'interface graphique, du serveur web embarqué ainsi que de la boucle principale du système.

Cette architecture améliore la réactivité du prototype mais complique le raisonnement sur le déroulement du programme. Les différentes tâches doivent partager certaines ressources et communiquer entre elles sans provoquer de conflits ou de comportements imprévisibles.

Afin de limiter cette complexité, une architecture orientée événements a été mise en place. Les différentes tâches ne réalisent que les opérations spécifiques à leur domaine de responsabilité et transmettent les événements qu'elles produisent à une file de messages (**queue**) FreeRTOS. Une machine à états centrale, exécutée dans la boucle principale, est ensuite chargée d'interpréter ces événements et de décider des actions à entreprendre.

Cette organisation présente plusieurs avantages. Les échanges entre les modules sont centralisés, les traitements critiques restent séquentiels et le comportement global du système est plus simple à comprendre et à maintenir malgré l'utilisation d'un environnement multitâche.

8.3 Une configuration pensée pour l'utilisateur

Lors des premières réflexions sur le projet, la configuration des serrures reposait sur une approche classique consistant à renseigner les paramètres directement dans le firmware avant sa compilation. Chaque modification du réseau Wi-Fi ou de la configuration du backend nécessitait alors une recompilation puis un nouveau flash du microcontrôleur.

Cette méthode s'est rapidement révélée peu adaptée à un système destiné à être installé sur plusieurs dispositifs. La configuration devenait fastidieuse et nécessitait systématiquement l'intervention d'une personne disposant des outils de développement.

Une autre approche a donc été imaginée : utiliser des **QR codes comme interface de configuration**. Lorsqu'une serrure est démarrée sans paramètres enregistrés, elle passe automatiquement en mode configuration et attend la lecture des QR codes contenant les informations nécessaires à son initialisation.

Cette solution transforme la caméra en véritable interface utilisateur. La configuration peut être réalisée directement sur le lieu d'installation sans connexion physique à l'ESP32 ni recompilation du firmware. Elle simplifie considérablement le déploiement du système tout en offrant une expérience d'installation plus intuitive.

8.4 Évolution du prototype matériel

Les premières versions du prototype ont été réalisées sur une breadboard, afin de faciliter les essais et les nombreuses modifications du câblage durant les phases de développement. Cette approche permettait de remplacer rapidement un composant ou de modifier les connexions au fur et à mesure de l'évolution du projet.

Une fois l'architecture matérielle validée, un montage plus robuste a été réalisé sur une plaque à pastilles. Les composants ont été soudés de manière permanente puis intégrés dans un boîtier en carton conçu spécifiquement pour être fixé sur une porte.

Bien que ce boîtier ne constitue pas une solution industrielle, il a permis de disposer d'un prototype fonctionnel, facilement transportable et suffisamment robuste pour réaliser les démonstrations du système dans des conditions proches d'une installation réelle.

8.5 Développement d'une couche graphique embarquée

L'interface graphique du prototype reposait initialement sur la bibliothèque U8g2, utilisée pour piloter l'écran OLED. Bien que cette bibliothèque remplisse parfaitement son rôle de pilote d'affichage, elle offre principalement des primitives graphiques de bas niveau.

Parallèlement au stage, plusieurs projets personnels de développement de jeux vidéo réalisés avec raylib ont permis de découvrir une approche plus moderne de la programmation graphique. Cette bibliothèque fournit non seulement une API de dessin homogène, mais également un ensemble d'outils mathématiques, notamment grâce au module raymath, facilitant la manipulation de vecteurs, de matrices et d'autres structures géométriques.

Dans le but de proposer une interface plus évoluée et de sortir des interfaces minimalistes souvent rencontrées sur les objets connectés, un portage partiel de raylib a été réalisé pour l'écran OLED utilisé par le prototype. Ce portage adapte les principales fonctions graphiques de la bibliothèque au pilote matériel existant et permet de bénéficier d'une couche d'abstraction plus riche tout en conservant les performances nécessaires au fonctionnement de l'ESP32.

Au-delà de l'aspect technique, cette initiative a permis de rendre le développement de l'interface plus agréable et plus modulaire. Elle constitue également une base réutilisable pour de futurs projets embarqués utilisant des écrans graphiques.

Les difficultés rencontrées durant le stage n'ont pas uniquement conduit à résoudre des problèmes techniques ; elles ont également permis de faire évoluer l'architecture du projet et de remettre en question certains choix initiaux. Les contraintes propres au développement embarqué ont nécessité de trouver un équilibre entre performances, simplicité de mise en œuvre et facilité de maintenance. Cette démarche d'amélioration continue a contribué à aboutir à un prototype plus robuste, plus cohérent et mieux adapté à une utilisation en conditions réelles.

9. Bilan et conclusion

9.1 Bilan du projet

Au cours de ce stage, un prototype complet de serrure connectée a été conçu et développé afin de répondre aux besoins de l'entreprise en matière de contrôle d'accès. L'objectif principal était de mettre en place une architecture permettant l'administration centralisée de serrures connectées, tout en proposant une solution simple à déployer et évolutive.

Le prototype réalisé intègre l'ensemble des composants nécessaires au fonctionnement du système. Le firmware embarqué sur ESP32 assure la gestion du matériel, la lecture des QR codes et la communication avec le backend. Celui-ci centralise la logique métier, gère l'authentification des utilisateurs et des serrures, valide les demandes d'accès et conserve l'historique des événements. Enfin, une interface web permet aux administrateurs de gérer les utilisateurs, les serrures et les différentes opérations du système.

Les objectifs définis au début du stage ont été atteints. Il est désormais possible de configurer une serrure grâce à des QR codes, d'autoriser ou de refuser un accès en fonction des informations enregistrées dans la base de données, de piloter l'ensemble du système depuis une interface d'administration et de consulter les événements générés par les différents dispositifs.

Au-delà des fonctionnalités développées, ce projet a également permis d'expérimenter plusieurs approches techniques avant d'aboutir à l'architecture retenue. Les contraintes propres au développement embarqué ont notamment conduit à faire évoluer certains choix initiaux, comme l'abandon des WebSockets sur les ESP32 au profit d'une communication HTTP initiée par les serrures. De la même manière, la mise en place d'une architecture événementielle reposant sur une machine à états et une file de messages a permis de simplifier l'organisation du firmware malgré l'utilisation de plusieurs tâches FreeRTOS.

Le prototype constitue aujourd'hui une base solide pouvant servir à de futurs développements. Plusieurs évolutions pourraient être envisagées, telles que l'intégration d'un mécanisme de mise à jour à distance du firmware (OTA), la conception d'un circuit imprimé dédié ou encore l'ajout de nouveaux moyens d'authentification et d'une gestion plus fine des permissions d'accès.

9.2 Bilan personnel

Ce stage m'a offert l'opportunité de participer au développement d'un projet complet, couvrant aussi bien les aspects logiciels que matériels. Contrairement aux projets réalisés dans le cadre de la formation, j'ai été amené à concevoir une solution destinée à répondre à un besoin concret, en tenant compte des contraintes techniques, des choix d'architecture et des méthodes de travail utilisées en entreprise.

Sur le plan technique, cette expérience m'a permis de renforcer mes compétences dans des domaines variés. J'ai approfondi mes connaissances en développement embarqué avec l'ESP32, ESP-IDF et FreeRTOS, tout en consolidant mes acquis en développement backend avec Node.js, Express et MongoDB. Le développement simultané du firmware, du backend et de l'interface d'administration m'a également permis d'appréhender la conception d'une architecture complète dans laquelle plusieurs technologies interagissent pour former un système cohérent.

Ce stage m'a également sensibilisé à l'importance des choix de conception. Plusieurs décisions prises au début du projet ont été remises en question afin de mieux répondre aux contraintes rencontrées. J'ai ainsi compris qu'une solution techniquement intéressante n'est pas nécessairement la plus adaptée, et qu'il est souvent préférable de privilégier une architecture simple, robuste et facilement maintenable.

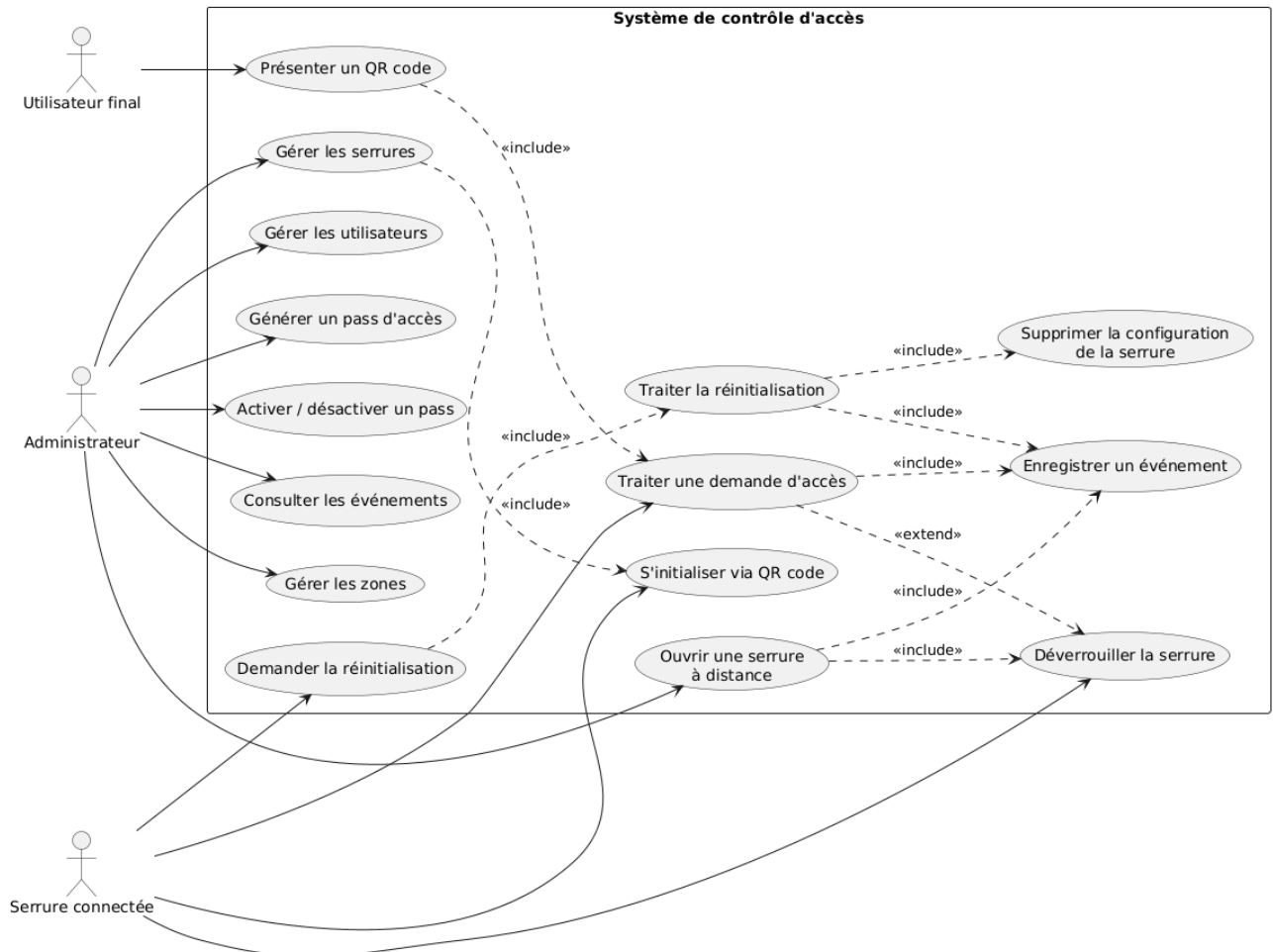
Au-delà des compétences techniques, cette expérience m'a permis de découvrir l'organisation d'un projet informatique en entreprise. L'utilisation quotidienne de Git, Forgejo et Planka, les échanges réguliers avec mon maître de stage ainsi que le fonctionnement inspiré des méthodes Agile m'ont permis de développer une meilleure autonomie et d'adopter une approche plus rigoureuse dans la conduite d'un projet.

Enfin, ce stage a confirmé mon intérêt pour le développement de systèmes combinant logiciel, électronique et systèmes embarqués. J'ai particulièrement apprécié la diversité des problématiques abordées, allant de la programmation bas niveau sur microcontrôleur jusqu'au développement d'une application web complète. La possibilité de concevoir un système de bout en bout, puis de le voir fonctionner concrètement sur un prototype, a constitué une expérience particulièrement enrichissante et motivante pour la suite de mon parcours.

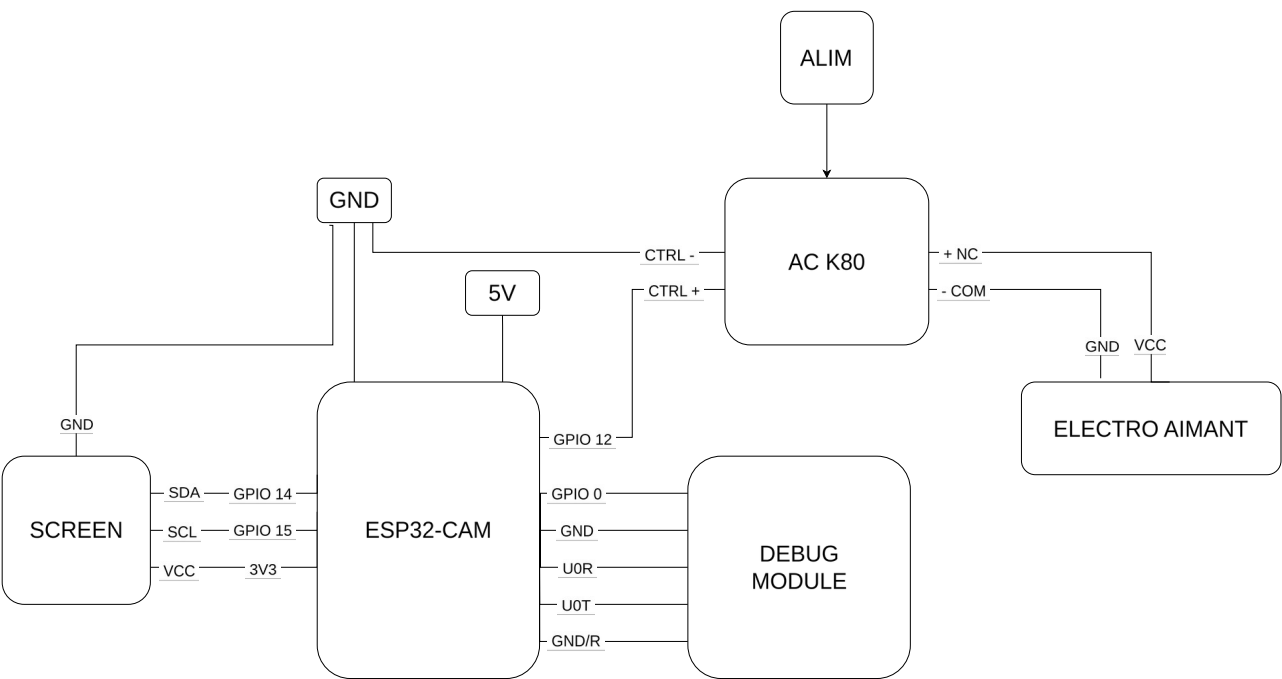
Le tableau d'auto-évaluation est situé après les annexes

10. Annexes

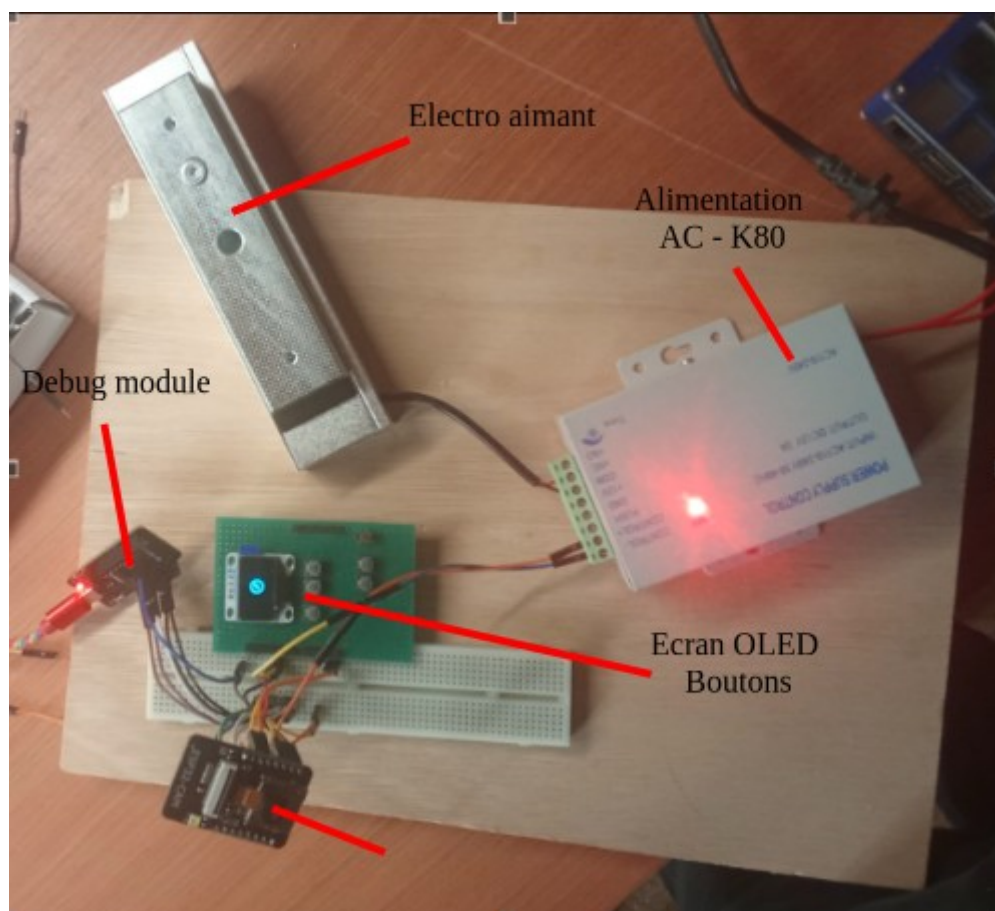
10.1 Diagramme de cas d'utilisations



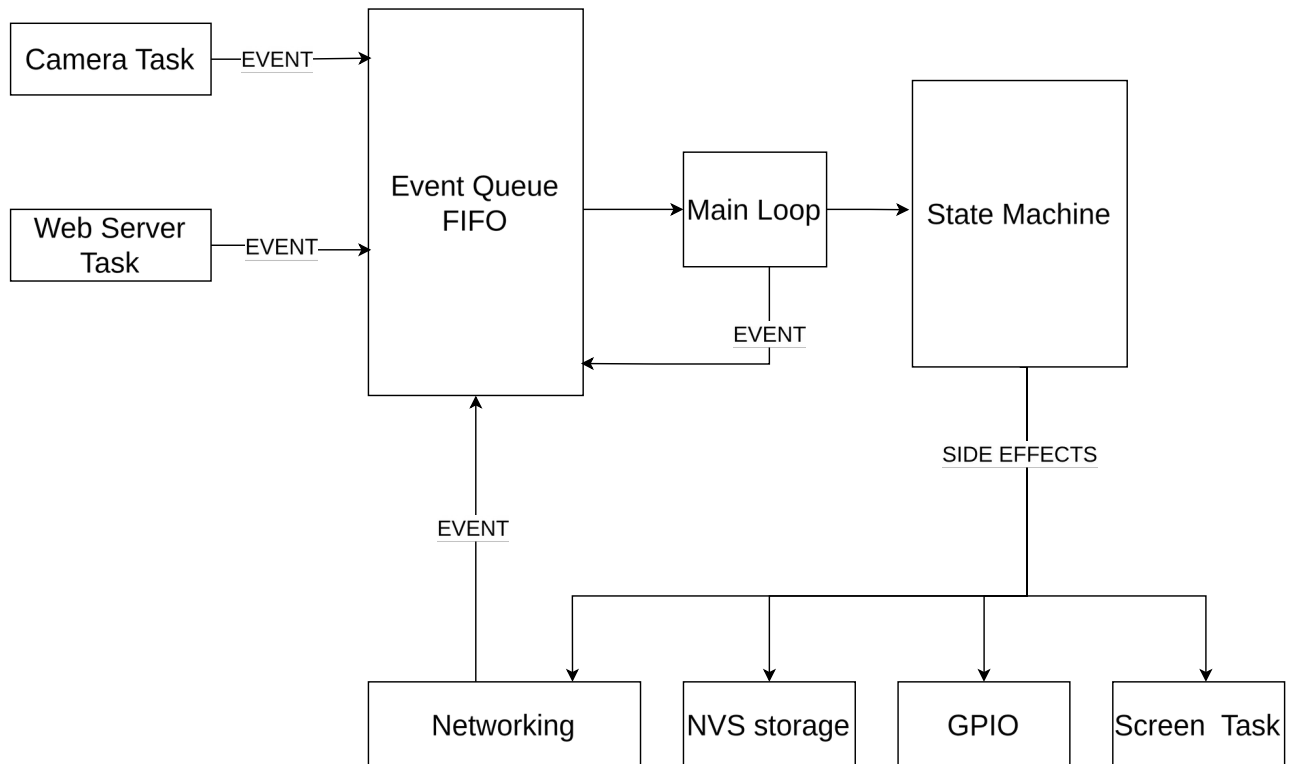
10.2 Schéma électrique



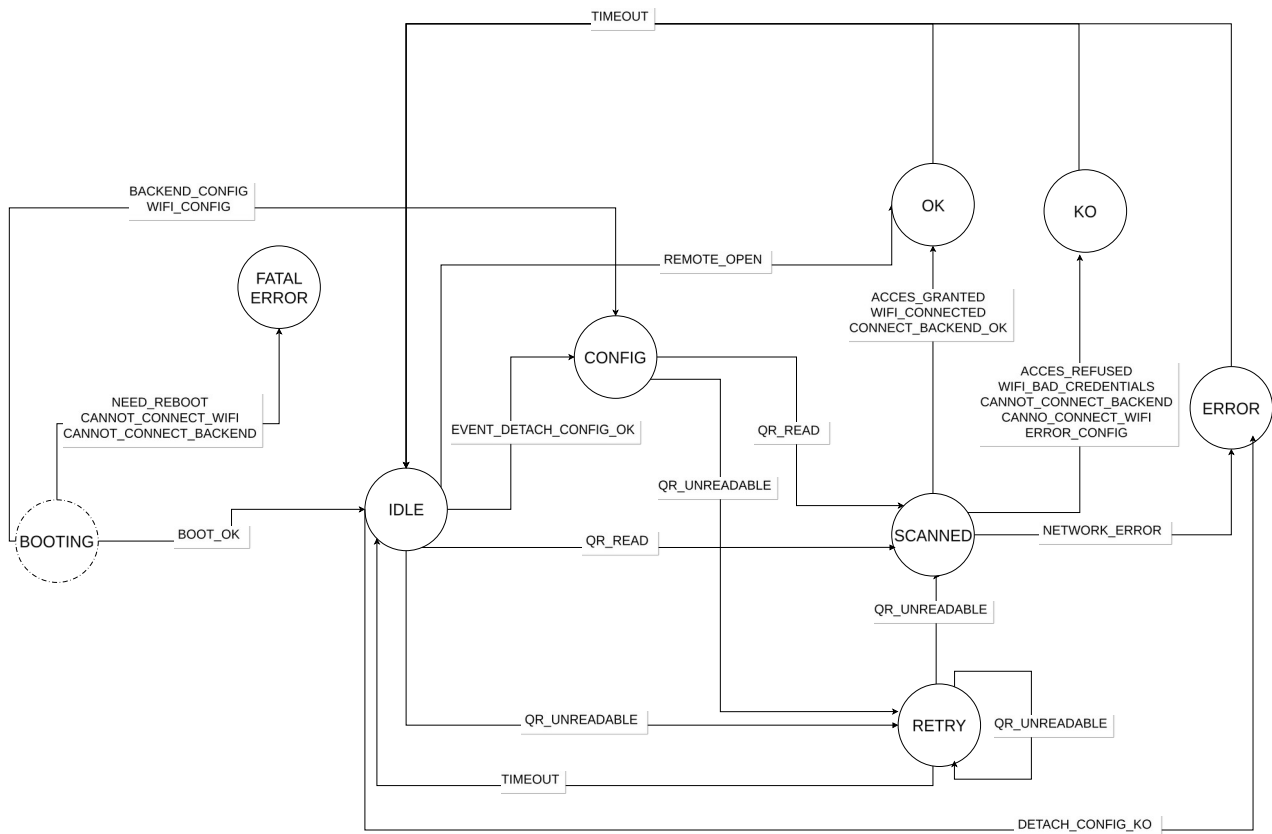
10.3 Photo du prototype



10.4 Diagramme architecture firmware

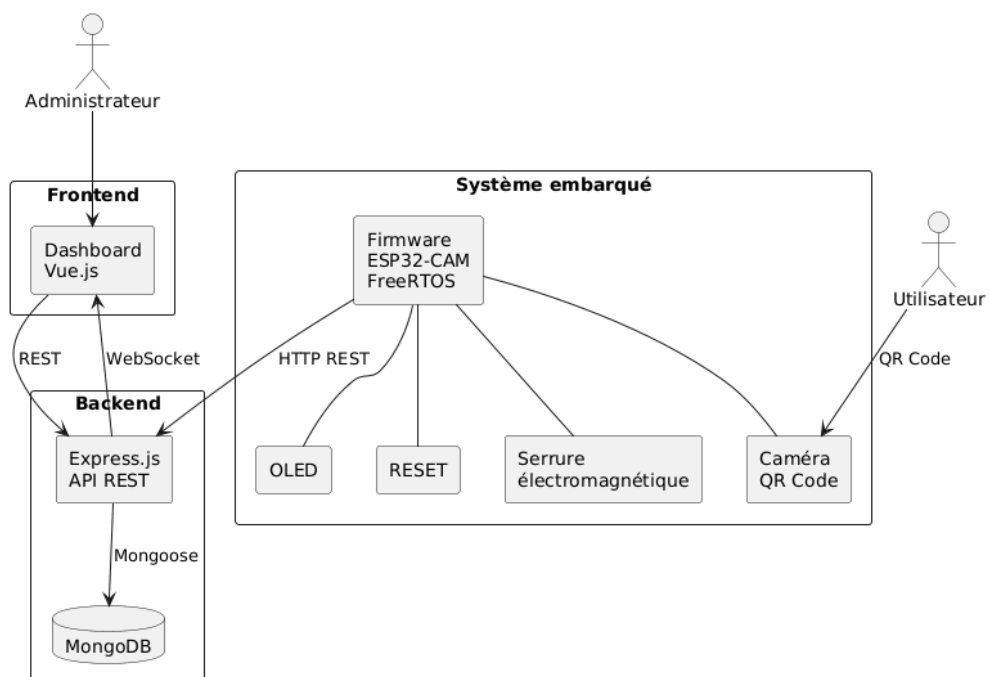


10.5 Diagramme state machine firmware

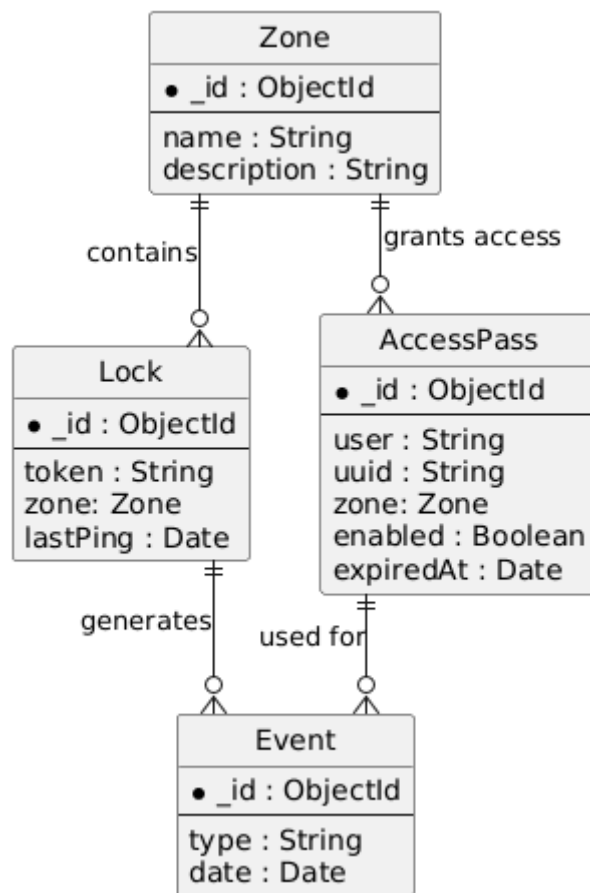


10.6 Diagramme architecture générale

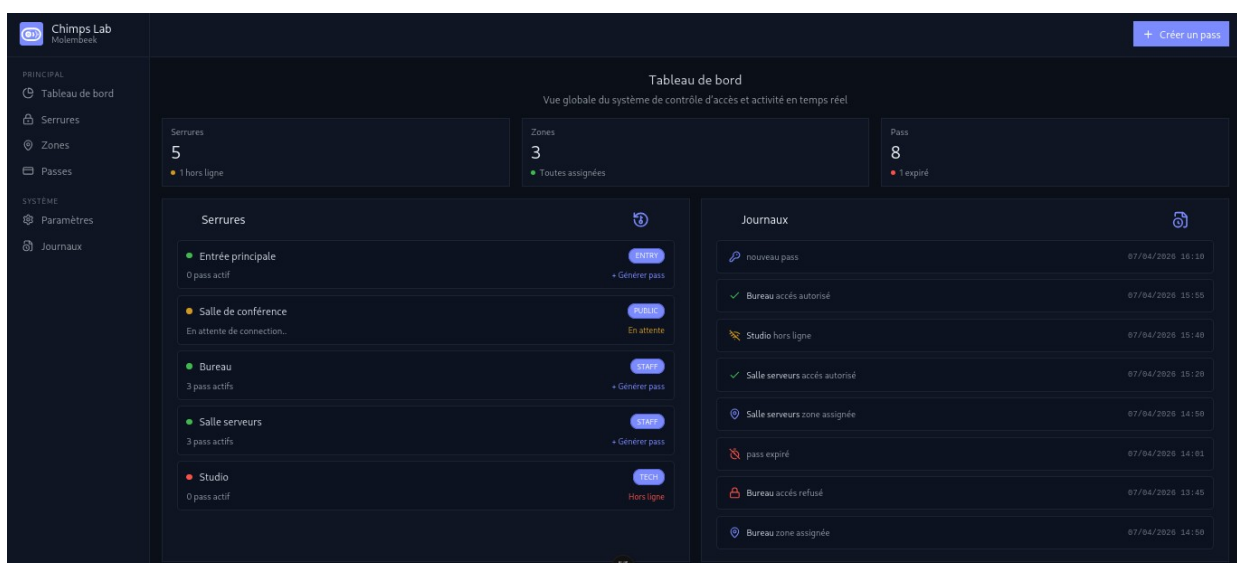
Architecture globale du système



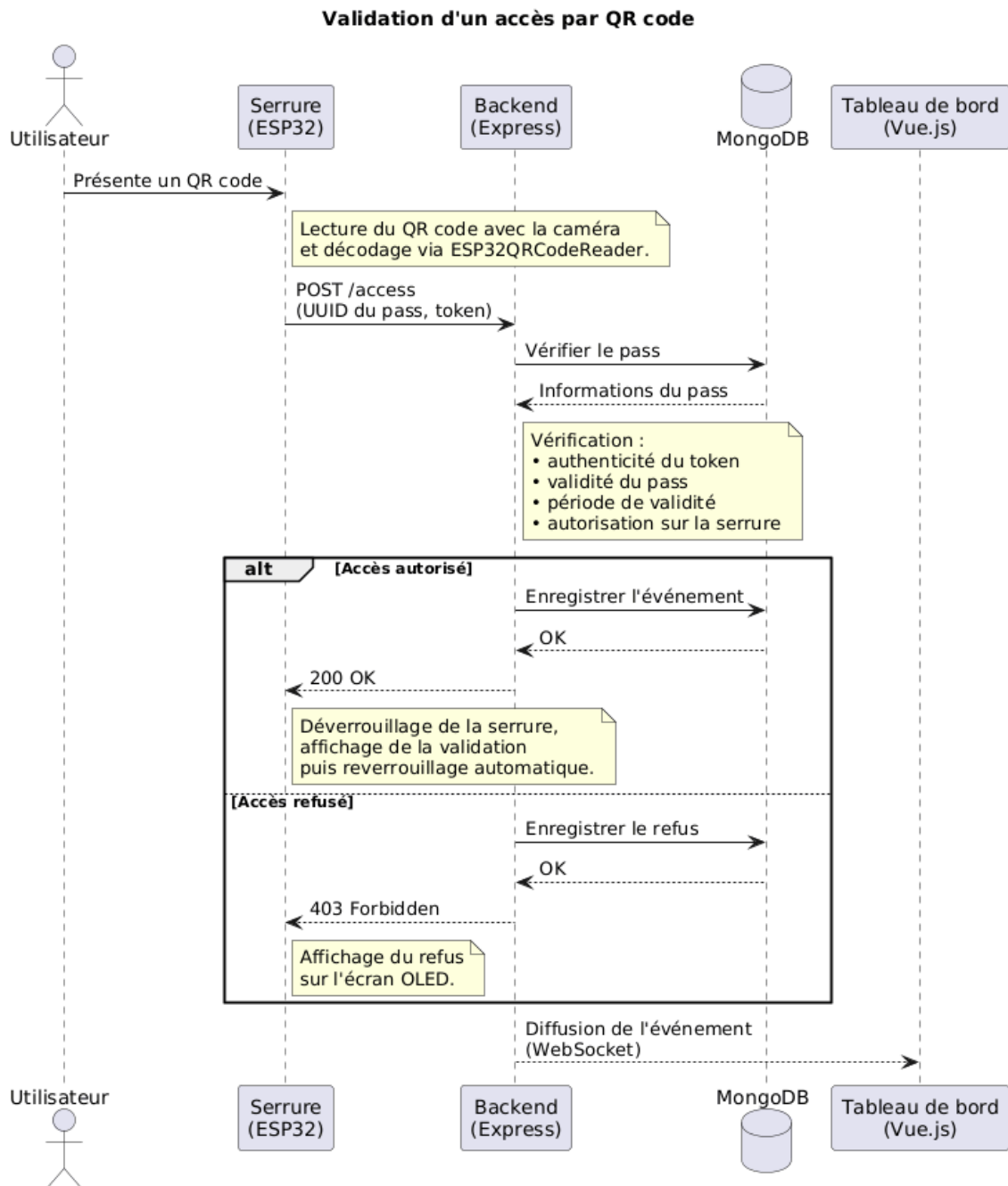
10.7 Diagramme ERD de la BDD



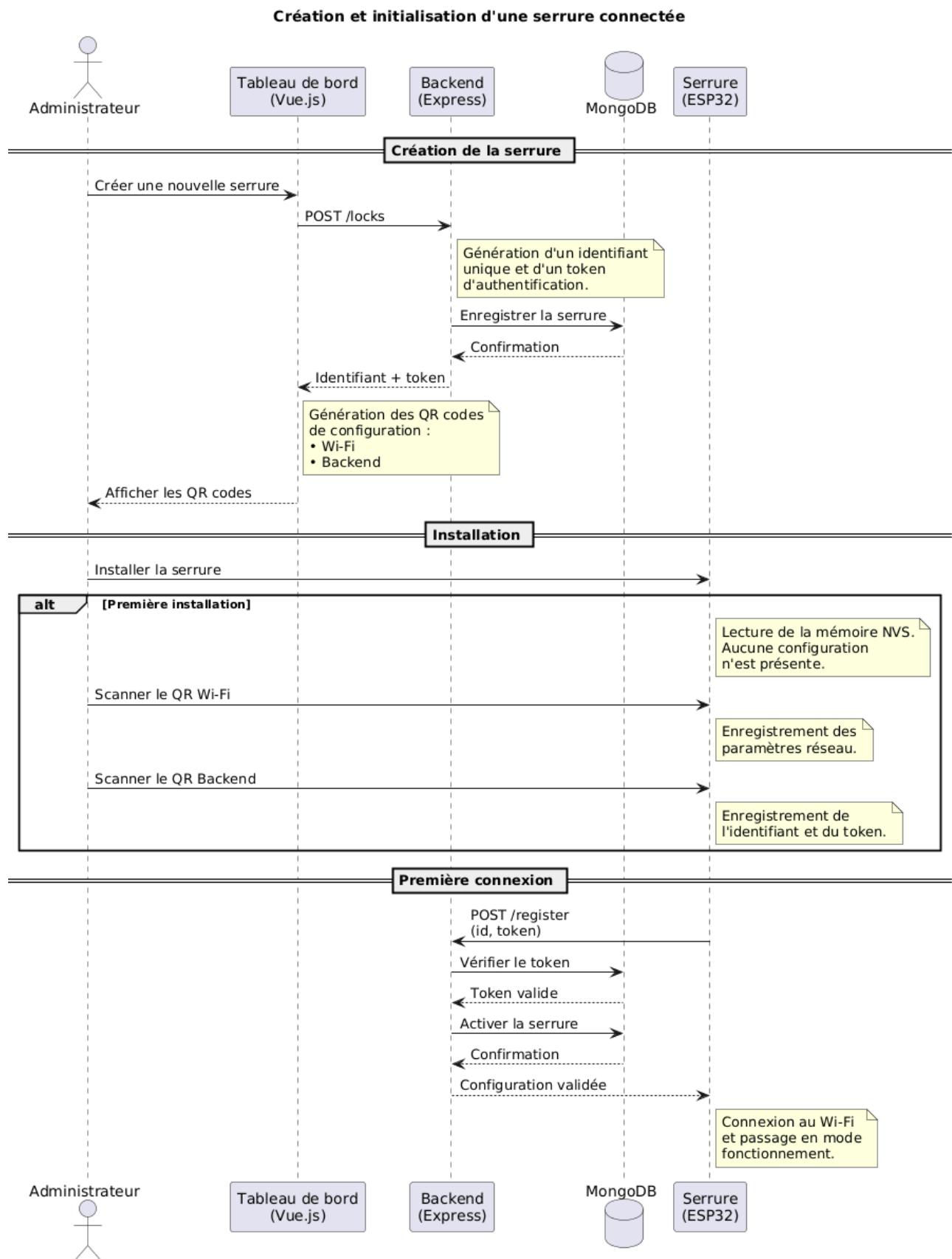
10.8 Visuel dashboard de la plateforme web



10.9 Diagramme de séquence 1



10.10 Diagramme de séquence 2



10.11 Diagramme de séquence 3

